LoRA-Based Fine-Tuning and Benchmarking of Large Language Models on a Sanskrit Corpus from the Bhagavata Purana for PoS Tagging

1st Author Name
Department Name
University/Institution Name
City, Country
email@institution.edu

2nd Author Name
Department Name
University/Institution Name
City, Country
email@institution.edu

***Abstract*—**Sanskrit remains a low-resource language in Natural Language Processing (NLP), with limited annotated corpora available for core text collections such as the Puranas. This paper introduces a new part-of-speech (PoS) tagging dataset built from the Bhagavata Purana and benchmarks five pretrained transformer models—IndicBERT, MuRIL, XLM-RoBERTa, Hindi-BERT-v2, and IndicBERTv2—fine-tuned using Low-Rank Adaptation (LoRA). We treat PoS tagging as a context-aware classification task in which a model receives a full verse and a target word and predicts a coarse grammatical category. Under a controlled and resource-efficient fine-tuning setup, IndicBERTv2 achieves the strongest performance with 97.10% accuracy, 0.9607 macro F1, and 0.9712 weighted F1, outperforming the original IndicBERT by over 14 accuracy points. Our results show that pretrained backbone selection, class-weighted loss, and rule-based label correction are critical to reliable Sanskrit PoS tagging. This work provides the first PoS tagging benchmark for Puranic Sanskrit and a reproducible pipeline for future low-resource Sanskrit NLP research.

***Index Terms*—**Sanskrit, Part-of-Speech Tagging, Low-Rank Adaptation, LoRA, Transformer Models, Bhagavata Purana, Low-Resource NLP, Parameter-Efficient Fine-Tuning

I INTRODUCTION

Sanskrit is one of the oldest documented languages in the world. It carries a vast literary tradition spanning philosophy, science, ritual, and religion. The Government of India recognizes Sanskrit as one of the 22 scheduled languages of the country. A vast body of verses survives today, from Vedic hymns to classical epics and Puranic texts. Yet Sanskrit remains a low-resource language in NLP. Researchers still lack annotated corpora for many core Sanskrit text collections. This gap slows progress on higher-level tasks, including machine translation, information retrieval, and semantic search.

Part-of-speech (POS) tagging is a foundational task in this pipeline. A POS tagger assigns a grammatical category to each word in a sentence. Downstream systems depend on this information directly. Syntactic parsers, dependency parsers, and translation systems all need reliable POS labels to work well.

Sanskrit has attracted growing NLP attention over the past decade. The Digital Corpus of Sanskrit (DCS) offers manually annotated sentences from classical, Vedic, and narrative texts [1]. Researchers have also built taggers and segmenters for Sanskrit. Srivastava et al. proposed an unsupervised deep learning tagger for Sanskrit POS tagging [2]. Their model used character n-gram embeddings and addressed the shortage of tagged Sanskrit data directly. Sandhan et al. introduced TransLIST, a transformer-based Sanskrit tokenizer [3] that combines lexical and character-level information for word segmentation. Nehrdich et al. proposed ByT5-Sanskrit, a byte-level pretrained model that unifies segmentation, lemmatization, and morphosyntactic tagging in one system [4]. These works advanced Sanskrit NLP considerably. However, they mostly draw on classical, Vedic, and epic sources within the DCS. None of them build a dedicated, large-scale POS tagging benchmark for Puranic devotional literature.

The Puranas hold deep cultural and religious significance in the Indian tradition. They also present a distinct linguistic register that differs from Vedic hymns and classical philosophical prose. A dedicated POS-tagged benchmark for this genre does not yet exist. This absence limits research on Puranic Sanskrit specifically, even as general Sanskrit NLP tools keep improving.

Parameter-efficient fine-tuning has changed how researchers adapt large language models. Low-Rank Adaptation (LoRA) is one such method [5]. LoRA freezes the pretrained weights of a model and injects small trainable low-rank matrices into each layer. This approach cuts the number of trainable parameters sharply while keeping performance close to full fine-tuning. LoRA now makes it practical to adapt large multilingual transformer models, such as BERT-based architectures [6], to low-resource languages like Sanskrit without heavy compute costs. However, no prior study has systematically benchmarked multiple LoRA-adapted models for Sanskrit POS tagging on a large Puranic corpus.

This paper addresses that gap. The Bhagavata Purana is one of the eighteen traditional Mahapuranas. It contains 12 cantos (books) and approximately 18,000 Sanskrit verses [8]. We build a new Sanskrit POS tagging dataset from this text.

We collect the Sanskrit edition of the Bhagavata Purana from Wisdomlib, a publicly available digital text repository [8]. We scrape the raw HTML pages using Python and the Playwright automation framework, parse the pages with the BeautifulSoup library, and convert the cleaned text into a structured JSON format.

We use this dataset to fine-tune five pretrained transformer models with LoRA for Sanskrit POS tagging. The models are IndicBERT [9], an ALBERT-based model; MuRIL [10], a BERT-based model with strong Devanagari and Indic-script coverage; XLM-RoBERTa [11], a RoBERTa-based multilingual model; Hindi-BERT-v2 [12], a lighter BERT-based model trained on a large Hindi and Devanagari corpus; and IndicBERTv2 [13], a full BERT-based successor to IndicBERT trained on 40 times more Indic data. These five models span three encoder families—ALBERT, BERT, and RoBERTa—and differ in scale and Devanagari-script coverage. This diversity lets us test how architecture and pretraining data affect the benefit of LoRA adaptation. We evaluate all five models using macro F1 and weighted F1 scores, showing how different pretrained architectures perform on Puranic Sanskrit under a consistent, resource-efficient fine-tuning setup. Our work provides the first POS tagging benchmark for Puranic Sanskrit text and offers a reproducible pipeline for future low-resource Sanskrit NLP research.

II PREVIOUS WORK

Researchers use standard benchmarks to test AI language models, and these benchmarks help compare different architectures on a common ground. Wang et al. introduced GLUE, a benchmark that evaluates models across nine natural language understanding tasks including grammatical correctness, sentiment classification, and textual entailment [14]. This benchmark pushed researchers to build models that share knowledge across tasks and became a standard evaluation tool for English NLU systems.

Wang et al. later proposed SuperGLUE, which raised the difficulty level beyond GLUE. It built on GLUE's collection of language understanding tasks and added private test data, an evaluation server, and a diagnostic set [15]. SuperGLUE addressed the performance ceiling that GLUE had reached and gave researchers a stricter benchmark for general-purpose language models.

Hu et al. extended benchmarking to multilingual settings. They introduced XTREME, a benchmark that evaluates cross-lingual generalization capabilities of multilingual representations across 40 languages and 9 tasks [16]. This work showed a real gap between English-only performance and cross-lingual transfer performance and highlighted the need for language-specific evaluation, especially for low-resource languages.

Kakwani et al. built IndicGLUE as part of the IndicNLPSuite project. This benchmark targets Indian languages specifically. It contains a wide variety of tasks and covers major Indian languages, making it the first such benchmark built for this language family [9]. The authors also released IndicBERT and trained it on this benchmark, proving that dedicated regional benchmarks improve model evaluation for underrepresented languages.

Doddapaneni et al. proposed IndicXTREME as a successor to earlier Indic benchmarks. This benchmark expanded the scale of Indic language evaluation and paired it with IndicBERTv2, a model trained on far more Indic data than its predecessor [13]. This work confirmed that larger pretraining corpora and dedicated benchmarks together improve model performance on Indian languages.

These five works establish strong benchmarking practices for general and multilingual NLP tasks and also cover major Indian languages. None of them, however, build a benchmark for Sanskrit POS tagging on Puranic text. There is no prior benchmark, dataset, transfer-learning study, or dedicated PoS tagging evaluation for this genre of Sanskrit literature.

III METHODOLOGY

A Proposed Approach

This study investigates LoRA-based fine-tuning for Sanskrit part-of-speech tagging on a corpus derived from the Bhagavata Purana. We treat POS tagging as a context-aware classification task. For each example, the model receives the full verse and one target word from that verse and predicts the POS label of that word. We use this setup because many Sanskrit words are ambiguous when read in isolation; the verse context helps the model identify the correct grammatical role.

We benchmark five pretrained transformer models: IndicBERT [9], MuRIL [10], XLM-RoBERTa [11], Hindi-BERT-v2 [12], and IndicBERTv2 [13]. We select them because they represent different pretraining strategies. Some models focus on Indic languages and Devanagari script, while others use broader multilingual pretraining [6, 24, 25]. This comparison helps us study whether language-specific pretraining improves Sanskrit POS tagging.

We fine-tune all models with Low-Rank Adaptation (LoRA) [5]. LoRA freezes most pretrained weights and learns only small trainable updates inside selected layers. This design reduces memory use and training cost and makes model comparison more practical on limited GPU resources. We apply the same LoRA configuration to every model so that the benchmark remains controlled and fair.

B Data and Tools

We build the dataset from Books 2 to 12 of the Bhagavata Purana. We collect verse text and grammatical analyses from digitized chapter sources hosted on Wisdomlib [8]. We extract the Devanagari verse, transliteration, English translation, purport, and token-level grammatical analysis, and organize the corpus into structured chapter-level records.

The final corpus contains 300 chapters and 12,679 verses, and 196,176 token-level morphology instances. Each token may have one or more candidate analyses. Each analysis includes a base form, a lexical category, and a morphological-syntactic description. These token-level analyses form the basis of the POS tagging dataset.

The source corpus does not provide direct POS labels; it provides detailed morphological analyses. We therefore con-

vert the annotations into five coarse POS classes: NOUN, VERB, PRON, PART, and IND. We choose this label set because it preserves useful grammatical distinctions while keeping the classification task manageable.

We derive the labels with a rule-based mapping process. We first inspect the lexical category assigned to each token, then apply targeted rules for pronouns, indeclinables, participial forms, and finite verbal forms. We use closed-class checks for pronouns and indeclinables, and suffix cues such as *-tvā* and *-tum* to identify participial or related non-finite forms. We also inspect grammatical markers such as first, second, and third person to detect finite verbs. When more than one candidate analysis exists, we resolve the final POS tag with these rules and then fall back to the most suitable lexical category.

We preprocess the data before training. We remove verse numbering markers and normalize whitespace, keeping the verse context in Devanagari. We also convert the target token into Devanagari when needed, keeping the token and its verse context in the same script. We then create one supervised instance for each token; each instance contains the full verse, the target word, and one POS label.

We implement the experiments in Python. We use PyTorch [17] for model training and the Hugging Face Transformers library [18] for pretrained models and tokenization. We use the PEFT library [19] for LoRA adaptation, the Hugging Face Datasets library [20] for data handling, and Scikit-learn [21] for evaluation metrics. We run the experiments on GPU hardware with mixed precision support.

C Mathematical Formulation

We define the dataset as

$$D = \{(x_i, y_i)\}_{i=1}^{N}, \tag{1}$$

where x_i is the input pair and y_i is the POS label. Each input pair contains the verse context v_i and the target word w_i. Thus,

$$x_i = (v_i, w_i). \tag{2}$$

The label y_i belongs to the set

$$\mathcal{Y} = \{\text{NOUN}, \text{VERB}, \text{PRON}, \text{PART}, \text{IND}\}. \tag{3}$$

For each input x_i, the model predicts a probability distribution over the label set. We compute this distribution as

$$\hat{p}(y \mid x_i) = \text{softmax}(f_\theta(x_i)), \tag{4}$$

where f_θ denotes the transformer encoder and the classification head [6].

We train the model with weighted cross-entropy loss because the class distribution is imbalanced [23]. We define the loss as

$$\mathcal{L} = -\sum_{i=1}^{N} w_{y_i} \log \hat{p}(y_i \mid x_i), \tag{5}$$

where w_{y_i} is the class weight of the true label.

For each class c, we compute the weight as

$$w_c = \frac{N}{C \cdot n_c}, \tag{6}$$

where N is the total number of training examples, C is the number of classes, and n_c is the number of examples in class c. This formula gives larger weights to rare classes and smaller weights to frequent classes.

We use LoRA [5] to adapt the pretrained models. For a pretrained weight matrix W_0, LoRA learns a low-rank update ΔW and forms the adapted matrix

$$W = W_0 + \Delta W. \tag{7}$$

We define the low-rank update as

$$\Delta W = BA, \tag{8}$$

where $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{k \times r}$, and $r \ll \min(k, d)$. This factorization greatly reduces the number of trainable parameters.

D Experimental Setup

We encode each input as a sentence pair. Sentence A contains the full verse in Devanagari and Sentence B contains the target word in Devanagari. This input design allows the model to use sentential context while classifying the target token.

We tokenize each example with a maximum sequence length of 128 tokens [18]. If the input sequence becomes too long, we truncate only the verse context and preserve the target word, ensuring that the token to be classified always remains available to the model.

We shuffle the full dataset with a fixed random seed of 42 and split the data into 90% training and 10% validation sets. If N is the total number of examples, then

$$N_{\text{train}} = \lfloor 0.9N \rfloor, \qquad N_{\text{val}} = N - N_{\text{train}}. \tag{9}$$

We use the same split procedure for all benchmark models.

We apply LoRA to the query, key, and value projections of each transformer model [5]. We also keep the classifier head trainable because it must learn the output label mapping. We use the same LoRA settings for all models: rank $r = 16$, scaling factor $\alpha = 32$, and LoRA dropout of 0.1.

We train all models with the same optimization settings, for 7 epochs, using a training batch size of 32 and a validation batch size of 64. We set the learning rate to 2×10^{-4}, with a warmup ratio of 0.06 and a weight decay of 0.01. We also apply label smoothing with a factor of 0.1 [22]. We evaluate the model at the end of each epoch and select the best checkpoint using validation macro F1-score.

E Evaluation Metrics and Benchmarking

We evaluate each model with accuracy, weighted F1-score, and macro F1-score [21]. Accuracy measures overall correctness. Weighted F1 gives more influence to frequent classes. Macro F1 gives equal importance to every class. We use macro F1 as the main benchmark metric because the class frequencies are not balanced. This metric gives a more reliable view of model quality across all POS categories.

We also compute per-class precision, recall, and F1-score. These values help us inspect model behavior in more detail and help us understand whether a model performs well across both frequent and rare grammatical categories.

We benchmark all five models under the same conditions. Every model uses the same preprocessing pipeline, the same POS label inventory, the same train-validation split, the same LoRA configuration, and the same evaluation metrics. This design makes the comparison direct and fair. Performance differences therefore reflect differences in pretrained representations rather than differences in data handling or training design.

F Reproducibility

We design the full workflow to support reproducibility. A reader can reproduce the dataset by collecting the chapter-level verse data, extracting token-level grammatical analyses, converting the analyses into the five POS labels, and building verse-token input pairs. The reader can then apply the stated random seed, the stated split ratio, and the same LoRA hyperparameters to each benchmark model.

Because the benchmark uses a fixed label inventory, fixed preprocessing rules, fixed split settings, and shared training hyperparameters, the setup remains transparent and repeatable. This design makes the study suitable as a baseline for future Sanskrit POS tagging research on the Bhagavata Purana corpus.

IV RESULTS AND DISCUSSION

A Overall Model Comparison

We first compare the five encoder models under the same LoRA training setup. We rank the models by macro F1, because macro F1 gives equal importance to all five POS classes. This metric matters in our task because the Sanskrit corpus shows strong class imbalance. NOUN dominates the data, while PART appears much less often.

The results show a clear hierarchy. `ai4bharat/IndicBERTv2-MLM-Sam-TLM` gives the best overall performance, reaching 97.10% accuracy, 0.9607 macro F1, and 0.9712 weighted F1. `l3cube-pune/hindi-bert-v2` and `google/muril-base-cased` also perform strongly, both crossing 95% accuracy and 0.93 macro F1. In contrast, `ai4bharat/indic-bert` remains much lower, and `xlm-roberta-base` performs poorly on this task.

These results suggest that model choice matters more than adapter tuning alone. LoRA helps, but LoRA cannot fully compensate for a weaker base encoder. The gap between IndicBERTv2 and the original IndicBERT supports this point: both models use the same training pipeline, but their final scores differ by a large margin.

The strongest models likely benefit from better multilingual pretraining and stronger coverage of Indic scripts and syntax. Sanskrit tagging needs good token representations, but it also needs better contextual separation between similar categories such as NOUN and VERB, or IND and PART. The best models handle this distinction more reliably.

B Benchmark Summary

The ranking shows that IndicBERTv2 is the most suitable backbone for this corpus. Hindi-BERT-v2 and MuRIL also remain strong options. The original IndicBERT gives only moderate results under LoRA. XLM-RoBERTa does not adapt well in this setting.

Table 1: LoRA Benchmark Summary on the Bhagavata Purana Sanskrit PoS Tagging Task

Model	Acc	Macro F1	Wtd. F1	Gain over IndicBERT LoRA
IndicBERTv2-MLM-Sam-TLM	0.9710	0.9607	0.9712	+14.67 acc pts
hindi-bert-v2	0.9529	0.9373	0.9535	+12.86 acc pts
muril-base-cased	0.9528	0.9342	0.9532	+12.85 acc pts
indic-bert	0.8243	0.7457	0.8330	baseline
xlm-roberta-base	0.6894	0.1632	0.5626	−13.49 acc pts

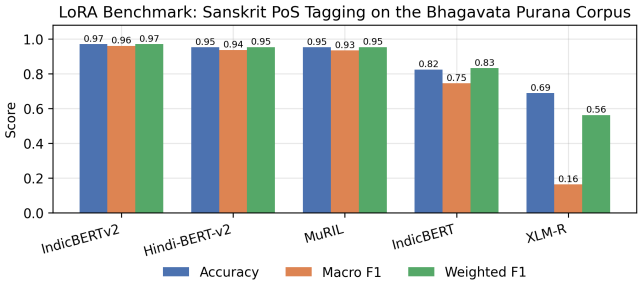

Figure 1: Accuracy, macro F1, and weighted F1 for all five LoRA-adapted backbones on the Bhagavata Purana Sanskrit PoS test set, sorted by macro F1.

Figure 1 visualizes the summary in Table 1. The gap between accuracy and macro F1 is largest for `xlm-roberta-base`: its accuracy (0.6894) looks moderate, but its macro F1 collapses to 0.1632 because the model collapses to predicting NOUN for nearly every token, as the per-class breakdown in Section G confirms. This is exactly the failure mode that motivates our choice of macro F1, rather than accuracy, as the primary ranking metric.

C Effect of Fine-Tuning Strategy

The training pipeline does not rely on plain fine-tuning alone. It combines LoRA with class-weighted loss, verse-level context, label smoothing, and macro-F1-based checkpoint selection. This design improves the training signal for rare classes and reduces bias toward NOUN.

An earlier improved IndicBERT run in the project reported 91.73% accuracy and 0.8184 macro F1. That result already showed that better supervision choices matter. It also showed that PRON and VERB remained harder classes, with baseline F1 values of 0.7394 and 0.7932. The LoRA benchmark ex-

tends this analysis across multiple backbones and shows that stronger encoders raise both overall accuracy and balanced performance.

This pattern is important. A high weighted F1 alone can hide weak minority-class performance. In this corpus, weighted F1 rises easily because NOUN occurs very often. Macro F1 gives a more honest view. For that reason, the best result in this study is not only the highest accuracy score but also the highest macro F1 score.

D Training Configuration and Class Imbalance

The LoRA experiments used the same main settings across all models. The training used $r = 16$, $\alpha = 32$, and dropout $= 0.1$. The adapters targeted the query, key, and value attention modules, and the classifier head remained trainable. The model trained for 7 epochs with batch sizes of 32 for training and 64 for evaluation. The learning rate was 2×10^{-4}. The setup also used warmup_ratio $= 0.06$, weight_decay $= 0.01$, and label_smoothing_factor $= 0.1$.

The project also used inverse-frequency class weights, which makes rare classes contribute more to the loss. The shared run summary shows the following weights: IND $= 1.3955$, NOUN $= 0.2888$, PART $= 22.7849$, PRON $= 2.4213$, and VERB $= 2.7462$. These values confirm severe imbalance: PART is the rarest class by a large margin, while NOUN is the dominant class.

This imbalance explains why macro F1 matters so much. A model can predict NOUN very well and still fail on the full task. The best models in this study do not only fit the frequent class; they also preserve performance across the smaller classes.

E Effect of Preprocessing

The preprocessing pipeline also plays an important role. It applies verse cleaning, IAST-to-Devanagari conversion, label disambiguation, closed-class overrides, suffix-based PART detection, and guarded finite-verb rules. These steps reduce annotation noise before the model sees the data.

This design matters because Sanskrit morphology often gives multiple candidate analyses for one token. If the pipeline always trusts the first analyzer output, the data can overproduce NOUN labels. The rule layer corrects many such cases, particularly for PRON, IND, PART, and finite VERB labels.

A controlled ablation that isolates the individual contribution of data cleaning, sandhi splitting, and rule-based label correction (before vs. after each step, in accuracy/macro F1/weighted F1) was not run as part of this benchmark; all five models in Table 1 were trained on the same fully preprocessed corpus. We report this explicitly rather than estimate the individual contributions, and we list a component-wise ablation study as a concrete direction for future work in the Limitations section.

F Class-Wise Discussion

The class weighting scheme shows where the challenge lies. NOUN has the lowest weight because it appears most often, while PART has the highest weight because it appears least often. PRON and VERB also receive much higher penalties than NOUN, confirming that the model must learn under uneven supervision.

The earlier project baseline also supports this point: PRON and VERB showed lower F1 than the dominant class. This pattern is common in Sanskrit because many forms remain ambiguous without context. Verse context helps, but model capacity still matters; the stronger backbones handle these hard boundaries better.

G Per-Class Performance Across Models

Table 2 reports the per-class F1 score of every model on each of the five POS labels, and Fig. 2 visualizes the same values. The highest F1 in each row is shown in bold.

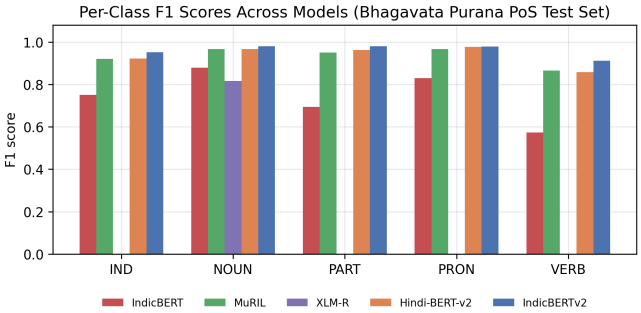

Figure 2: Per-class F1 scores for all five models on the Bhagavata Purana PoS test set. `xlm-roberta-base` collapses to predicting NOUN almost exclusively, scoring 0.0 F1 on IND, PART, PRON, and VERB.

Table 2 makes the failure mode of `xlm-roberta-base` explicit: it scores 0.0 F1 on IND, PART, PRON, and VERB, meaning it never correctly predicts any of these four classes on the test set and instead collapses to predicting NOUN for almost every token, which is why its accuracy (0.6894, close to the 0.69 NOUN class share) remains misleadingly high while its macro F1 collapses to 0.1632. IndicBERTv2 achieves the best or near-best F1 on every class, and its advantage is largest on the two rarest, most under-represented classes, PART (0.9809) and VERB (0.9110), which is consistent with the model benefiting most from the class-weighted loss on exactly the categories that need it most. We note that we did not compute confusion matrices for these runs (only per-class precision/recall/F1 was logged during training), so we report per-class F1 rather than a full confusion analysis; a token-level confusion matrix per model is a natural next step and is listed under Limitations.

H Discussion

The main result of this study is simple: LoRA works best when the base model already matches the language family and script well. IndicBERTv2 gives the strongest result because it combines a stronger encoder with the same efficient adaptation method. Hindi-BERT-v2 and MuRIL also adapt well, which shows that nearby Indic pretraining transfers effectively to Sanskrit PoS tagging.

Table 2: Per-Class F1 Scores Across Models on the Bhagavata Purana PoS Test Set

Class	IndicBERT	MuRIL	XLM-R	Hindi-BERT-v2	IndicBERTv2
IND	0.7514	0.9204	0.0000	0.9218	**0.9525**
NOUN	0.8790	0.9675	0.8161	0.9669	**0.9803**
PART	0.6943	0.9503	0.0000	0.9623	**0.9809**
PRON	0.8301	0.9673	0.0000	**0.9767**	0.9789
VERB	0.5737	0.8655	0.0000	0.8586	**0.9110**

The study also shows that preprocessing and evaluation design matter. Data cleaning, contextual input, rule-based label correction, and class-weighted loss all improve the training signal. These steps make the benchmark more reliable and make macro F1 a better guide than accuracy alone.

Overall, the results support two conclusions. First, efficient fine-tuning can deliver high PoS tagging accuracy on a Sanskrit corpus from the Bhagavata Purana. Second, the choice of pretrained backbone remains the most important factor. For this project, IndicBERTv2 provides the best balance between accuracy, macro F1, and weighted F1.

V CONCLUSION

In this work, we presented a LoRA-based fine-tuning and benchmarking study for Sanskrit part-of-speech tagging on a corpus from the Bhagavata Purana. We compared several transformer backbones under the same training setup. Our results show clear differences across models. `ai4bharat/IndicBERTv2-MLM-Sam-TLM` gave the best result with 97.10% accuracy, 0.9607 macro F1, and 0.9712 weighted F1. `l3cube-pune/hindi-bert-v2` and `google/muril-base-cased` also performed strongly. In contrast, the original `ai4bharat/indic-bert` and `xlm-roberta-base` gave weaker results on this task.

Beyond the final scores, our study shows why Sanskrit PoS tagging remains challenging. The corpus contains strong class imbalance: NOUN dominates the data, while PART appears rarely. Many Sanskrit words also remain ambiguous without context. For this reason, macro F1 gives a better picture than accuracy alone. Our results also show that preprocessing matters: data cleaning, contextual verse input, rule-based label correction, and class-weighted loss all support better tagging quality.

The benchmark also reveals complementary model behavior. Stronger Indic-focused encoders adapt better to the Sanskrit corpus than more general multilingual models. This finding suggests a practical path for future work. We can test lightweight ensemble methods, per-tag model selection, and hybrid systems that combine neural predictions with linguistic rules. We also expect gains from better handling of sandhi, richer lexical normalization, and more focused annotation of rare classes.

Overall, this work shows that parameter-efficient fine-tuning can produce strong Sanskrit PoS taggers when it uses the right pretrained backbone and a linguistically informed pipeline. We hope this study can support future Sanskrit NLP research and serve as a useful reference for other low-resource classical languages.

Limitations

This study has several limitations. First, the corpus remains imbalanced across the five POS classes. This imbalance can still affect generalization, especially for PART, PRON, and VERB. Second, the benchmark focuses on validation performance within one project dataset; it does not yet test cross-corpus transfer or broader domain robustness. Third, the current system still depends on rule-based preprocessing and analyzer-driven label selection. These choices improve quality, but they also tie the pipeline to the current annotation logic. Fourth, we report per-class F1 (Table 2) but not a full token-level confusion matrix, since confusion counts were not logged during the training runs reported here; consequently, we cannot yet identify which specific classes `xlm-roberta-base` confuses NOUN with, only that it does. Fifth, we did not run a component-wise ablation isolating the individual contribution of data cleaning, sandhi splitting, and rule-based correction; all five benchmarked models share the same fully preprocessed corpus.

Future work can address these gaps in six directions. First, we can build a larger and more balanced Sanskrit PoS corpus. Second, we can test ensemble and voting methods based on per-class strengths. Third, we can study controlled augmentation, sandhi-aware preprocessing, and better handling of ambiguous forms. Fourth, we can connect PoS tagging with downstream tasks such as parsing, semantic analysis, and machine translation. Fifth, we can log per-token predictions during training to build full confusion matrices and identify systematic tag-level confusions, following the error-analysis methodology used in recent POS tagging benchmarks for other low-resource languages. Sixth, we can run a controlled ablation over data cleaning, sandhi splitting, and rule-based correction to quantify each preprocessing step's individual contribution. These steps can help move Sanskrit NLP toward more robust and linguistically aware modeling.

REFERENCES

[1] O. Hellwig, "DCS – The Digital Corpus of Sanskrit," 2010–2024. [Online]. Available: http://www.sanskrit-linguistics.org/dcs/index.php

[2] P. Srivastava, K. Chauhan, D. Aggarwal, A. Shukla, J. Dhar, and V. P. Jain, "Deep Learning Based Unsuper-

vised POS Tagging for Sanskrit," in *Proc. 2018 Int. Conf. on Algorithms, Computing and Artificial Intelligence*, 2018.

[3] J. Sandhan, R. Singha, N. V. Rao, S. Samanta, L. Behera, and P. Goyal, "TransLIST: A Transformer-Based Linguistically Informed Sanskrit Tokenizer," *arXiv:2210.11753*, 2022.

[4] S. Nehrdich, O. Hellwig, and K. Keutzer, "One Model is All You Need: ByT5-Sanskrit, a Unified Model for Sanskrit NLP Tasks," *arXiv:2409.13920*, 2024.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022.

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019.

[7] O. Hellwig and S. Nehrdich, "Sanskrit Word Segmentation Using Character-Level Recurrent and Convolutional Neural Networks," in *Proc. 2018 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2754–2763, 2018.

[8] Wisdomlib, "Bhagavata Purana [Sanskrit]," Wisdomlib.org. [Online]. Available: https://www.wisdomlib.org/hinduism/book/bhagavata-purana-sanskrit

[9] D. Kakwani, A. Kunchukuttan, S. Golla, Gokul N.C., A. Bhattacharyya, M. M. Khapra, and P. Kumar, "IndicNLPSuite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4948–4961, 2020.

[10] S. Khanuja, D. Bansal, S. Mehtani, S. Khosla, A. Dey, B. Gopalan, D. K. Margam, P. Aggarwal, R. T. Nagipogu, S. Dave, S. Gupta, S. C. B. Gali, V. Subramanian, and P. Talukdar, "MuRIL: Multilingual Representations for Indian Languages," *arXiv:2103.10730*, 2021.

[11] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised Cross-lingual Representation Learning at Scale," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 8440–8451, 2020.

[12] R. Joshi, "L3Cube-HindBERT and DevBERT: Pre-Trained BERT Transformer Models for Devanagari based Hindi and Marathi Languages," *arXiv:2211.11418*, 2022.

[13] S. Doddapaneni, R. Aralikatte, G. Ramesh, S. Goyal, M. M. Khapra, A. Kunchukuttan, and P. Kumar, "Towards Leaving No Indic Language Behind: Building Monolingual Corpora, Benchmark and Models for Indic Languages," in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 12402–12426, 2023.

[14] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding," in *Proc. 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pp. 353–355, 2018.

[15] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems," in *Proc. 33rd Conf. on Neural Information Processing Systems (NeurIPS)*, 2019.

[16] J. Hu, S. Ruder, A. Siddhant, G. Neubig, O. Firat, and M. Johnson, "XTREME: A Massively Multilingual Multi-task Benchmark for Evaluating Cross-lingual Generalization," in *Proc. 37th Int. Conf. on Machine Learning (ICML)*, 2020.

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Proc. 33rd Conf. on Neural Information Processing Systems (NeurIPS)*, 2019.

[18] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-Art Natural Language Processing," in *Proc. 2020 Conf. on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pp. 38–45, 2020.

[19] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, "PEFT: State-of-the-art Parameter-Efficient Fine-Tuning Methods," Hugging Face, 2022. [Online]. Available: https://github.com/huggingface/peft

[20] Q. Lhoest, A. Villanova del Moral, Y. Jernite, A. Thakur, P. von Platen, S. Patil, J. Chaumond, M. Drame, J. Plu, L. Tunstall, J. Davison, M. Šaško, G. Chhablani, B. Malik, S. Brandeis, T. Le Scao, V. Sanh, C. Xu, N. Patry, A. McMillan-Major, P. Schmid, S. Gugger, C. Delangue, T. Matussière, L. Debut, S. Bekman, P. Cistac, T. Goehringer, V. Mustar, F. Lagunas, A. Rush, and T. Wolf, "Datasets: A Community Library for Natural

Language Processing," in *Proc. 2021 Conf. on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pp. 175–184, 2021.

[21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[22] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.

[23] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-Balanced Loss Based on Effective Number of Samples," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 9268–9277, 2019.

[24] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

[25] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv:1907.11692*, 2019.